.NET & AZURE MEETUP ŠTAJERSKA

THURSDAY, JANUARY 29th · 18:30 CET

ONLINE

Dependency Injection

How much do we really know it?

NICOLA PARO

A Real Life Story…

```
using System.Configuration;
using System.Data.SqlClient;


public class OrderService
{
    public void PlaceOrder()
    {
        var connectionString = ConfigurationManager.ConnectionStrings["OrdersDb"].ConnectionString;

        // lots of lines of business logic here

        using var connection = new SqlConnection(connectionString);
        connection.Open();


        var command = connection.CreateCommand();
        command.CommandText = "INSERT INTO Orders (...) VALUES (...)";
        command.ExecuteNonQuery();


        var smtpClient = new SmtpClient("smtp.company.local");
        smtpClient.Send("noreply@company.com", "admin@company.com", "Order placed", "");
    }
}
```

A Real Life Story…

```
using System.Confi
using System.Data.


public class Order
{
    public void Pl
    {
        var connec                                  ecti                                    ;

        // lots of lines of logic here

        using var                                   ionS
        connection


        var comman
        command.Co                                  VALU
        command.Ex


        var smtpCl                                  cal"
        smtpClient                                  mpar
    }
}
```

How do I test it?

How can I change it?

How do I maintain it

How to extend it?

The problem behind…

A class that directly creates its own dependencies manages **what it uses** instead of **what it needs**

You can't easily change the implementation code

Unit testing becomes difficult or impossible

A single change propagates throughout the code

So, what is Dependency Injection?

It is a technique and design pattern where objects do not create their own dependencies, but dependencies are provided from the outside.

Depend on abstractions, not implementations!

So, what is Deep Injection?

So, what is Dependency Injection?

```
public interface IEngine { }
public class VolksWagenEngine : IEngine { }
public class BMWEngine : IEngine { }
public class FerrariEngine : IEngine { }


public interface IFarmingTractor { }
public class FarmingTractor(IEngine engine) : IFarmingTractor { }


public class Farmer(IFarmingTractor tractor) { }
```

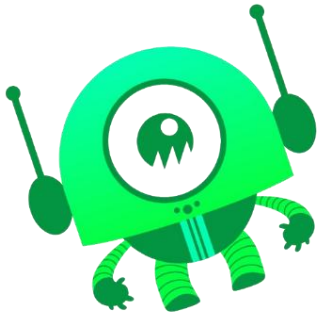
What is a DI container

A DI Container is a library that automatically manages dependencies between objects in an application.

- ✓ Resolves dependencies
- ✓ Manages dependencies lifecycles
- ✓ Supports complex constructors
- ✓ Facilitates testing and maintenance
- ✓ Reduces coupling

- ✕ Is not a Service Locator
- ✕ Doesn’t decide app logic
- ✕ Does not replace good design

DI containers



Autofac

Castle Windsor

DI containers

Microsoft.Extensions.DependencyInjection

- Integrated DI container in .NET
- Consistent approach across different frameworks
- Lightweight and fast
- Works with ASP.NET Core, Windows Services, Console apps, Windows Forms, etc.
- **netstandard 2.0**
  - Works with .NET 8, 9, 10 and .NET Framework

IServiceCollection and IServiceProvider

IServiceCollection defines how to build the dependency tree

IServiceProvider is responsible for:

- Creating object instances
- Injecting dependencies
- Managing object lifecycles according to the lifetime specified during registration in IServiceCollection

Service Lifetime

Singleton	Scoped	Transient
• Single instance for the entire application • Must be thread-safe	• Instance per scope*	• New instance every time it is requested • Designed for lightweight, stateless services

*In AspNetCore:

- For each HTTP request, a scope is automatically created by the framework.
- It is not true that a scope always corresponds to an HTTP request.

How does injection happen

Service injection mainly happens via Constructor Injection, where dependencies are injected through constructor parameters:

- Explicit dependencies
- Prevents situations with partially initialised instances

How do I register a dependency

Abstraction + Implementation

```
var services = new ServiceCollection();
```

Lifetime	Abstraction	Implementation

```
services.AddSingleton<IMyService1, MyService1Implementation>();
services.AddScoped<IMyService2, MyService2Implementation>();
services.AddTransient<IMyService3, MyService3Implementation>();
```

How do I register a dependency

Abstraction + Implementation

```
var services = new ServiceCollection();



services.AddSingleton<IMyService1, MyService1Mock>();

services.AddScoped<IMyService2, MyService2Mock>();

services.AddTransient<IMyService3, MyService3Implementation>();
```

How do I register a dependency

Only implementation

```
var services = new ServiceCollection();



services.AddSingleton<MyService1Implementation>();

services.AddScoped<MyService2Implementation>();

services.AddTransient<MyService3Implementation>();
```

How do I register a dependency

Abstraction + implementationFactory

```
var services = new ServiceCollection();

services.AddSingleton<IMyService1>(sp => new MyService1Implementation());
services.AddScoped<IMyService2>(sp => new MyService2Implementation());
services.AddTransient<IMyService3>(sp => new MyService3Implementation());
```

How do I register a dependency

implementationFactory only

```
var services = new ServiceCollection();

services.AddSingleton(sp => new MyService1Implementation());

services.AddScoped(sp => new MyService2Implementation());

services.AddTransient(sp => new MyService3Implementation());
```

How do I register a dependency

When does it make sense to use a factory

```
public interface IMyServiceA { }
public interface IMyServiceB { }

public class MyServiceAImplementation(string name) : IMyServiceA { }
public class MyServiceBImplementation(IMyServiceA serviceA, string name) : IMyServiceB { }


services.AddSingleton<IMyServiceA>(
    sp => new MyServiceAImplementation("ServiceAName"));

services.AddSingleton<IMyServiceB> (
    sp => new MyServiceBImplementation(sp.GetRequiredService<IMyServiceA>(), "ServiceBName"));
```

How do I register a dependency

When a service has really lots of dependencies…

```
public class MyService(IMyServiceA serviceA, IMyServiceB serviceB
    , IMyServiceC serviceC, IMyServiceD serviceD
    , IMyServiceE serviceE) : IMyService { }

services.AddSingleton<IMyServiceA, MyServiceAImplementation>();
services.AddSingleton<IMyServiceB, MyServiceBImplementation>();
services.AddSingleton<IMyServiceC, MyServiceCImplementation>();
services.AddSingleton<IMyServiceD, MyServiceDImplementation>();
services.AddSingleton<IMyServiceE, MyServiceEImplementation>();
services.AddSingleton<IMyService, MyService>();
```

How do I register a dependency

When a service has really lots of dependencies, but I need a factory…

```
public class MyService(IMyServiceA serviceA, IMyServiceB serviceB
    , IMyServiceC serviceC, IMyServiceD serviceD
    , IMyServiceE service, string someString) : IMyService { }

services.AddSingleton<IMyServiceA, MyServiceAImplementation>();
services.AddSingleton<IMyServiceB, MyServiceBImplementation>();
services.AddSingleton<IMyServiceC, MyServiceCImplementation>();
services.AddSingleton<IMyServiceD, MyServiceDImplementation>();
services.AddSingleton<IMyServiceE, MyServiceEImplementation>();
services.AddSingleton<IMyService>(sp => new MyService(
    sp.GetRequiredService<IMyServiceA>(), sp.GetRequiredService<IMyServiceB>(),
    sp.GetRequiredService<IMyServiceC>(), sp.GetRequiredService<IMyServiceD>(),
    sp.GetRequiredService<IMyServiceE>(),"SomeStringValue"
));
```

How do I register a dependency

When a service has really lots of dependencies, but I need a factory…

```
public class MyService(IMyServiceA serviceA, IMyServiceB serviceB
    , IMyServiceC serviceC, IMyServiceD serviceD
    , IMyServiceE serviceE) : IMyService { }

services.AddSingleton<IMyServiceA, MyServiceAImplementation>();
services.AddSingleton<IMyServiceB, MyServiceBImplementation>();
services.AddSingleton<IMyServiceC, MyServiceCImplementation>();
services.AddSingleton<IMyServiceD, MyServiceDImplementation>();
services.AddSingleton<IMyServiceE, MyServiceEImplementation>();
services.AddSingleton<IMyService>(sp =>
    ActivatorUtilities.CreateInstance<MyService>(sp, "SomeStringValue");
```

ActivatorUtilities.CreateInstance comes to rescue!

How does injection happen

Normally, the DI container automatically creates instances by resolving all dependencies from the registered service.

However, sometimes we may want to create an instance of a class where only some constructor parameters are handled by the container, while others are passed explicitly.

- The arguments that cannot be resolved by the ServiceProvider are supplied to the constructor in the same order as they are provided.
- The public constructor with the highest number of parameters that can be satisfied (combining resolved services + provided parameters) is chosen.
- If no constructor is compatible → InvalidOperationException.
- Useful when you don't want to register a type in the serviceCollection, but still want to leverage DI for some of its dependencies.

But now I have some questions:

How do I use this in my legacy application?

Demo

Hidden traps

Service Locator

Injecting IServiceProvider everywhere

- Hides real dependencies and makes code less clear.

Captive Dependency

- Singleton depending on a Scoped or Transient service

Using too many Singletons

- Can cause memory leaks
- Shared state between multiple services

Some tips / how to avoid pain

Prefer constructor injection

Register services near the composition root

Small services focused on their goal

- OrderRepository manages orders, it does not create an SmtpClient to send an email

Clear abstractions

- IOrderRepository does not define a SendOrderConfirmationEmail method

Avoid static dependencies

- JsonSerializer
- Console

Wrapping up

Dependency Injection solves the problem of direct dependencies

It focuses on abstractions

Microsoft.Extensions.DependencyInjection is simple and powerful

Let’s get in touch!

Nicola Paro

Solution Architect | Microsoft MVP

linkedin.com/in/nicolaparo

github.com/nicolaparo

MVP
Microsoft®
Most Valuable
Professional

WE LIKE CI/CD

Continuous Improvement – Continuous Discussions

Share your feedback, opinions, suggestions with us!

WE ARE ON DISCORD!

https://discord.gg/K6F7pdP4xq

Hvala!